

✓ ES6 Classes and OOP

OOP → Objected - oriented programming -

✓ # class , object , properties + method

blueprint for creating object

```
class User {  
  constructor (name, age) {  
    this.name = name  
    this.age = age  
  }  
  greet() {  
    console.log (this.name)  
  }  
}
```

this → Always refers to current ~~class~~ object

1 four main pillar of oops

~① Inheritance
 ② Encapsulation
 ③ Polymorphism
 ④ Abstraction



```
// const user1 = {
//   name : "Satyam",
//   greet : function(){
//     console.log("Hii, I'm Satyam")
//   }
// }
// const user2 = {
//   name : "Shivam",
//   greet : function(){
//     console.log("Hii, I'm Shivam")
//   }
// }
```

```
// class => Blueprint for creating object
// class User {
//   // constructor => automatically runs when we create an
//   // object
//   constructor (name,age){
//     this.name = name
//     this.age = age
//   }
//   greet(){
//     console.log("Hii i'm greet")
//   }
// }
// const user1 = new User("satyam",23)
// console.log(user1)
// user1.greet()
// Inheritance => One class can inherit properties and methods
// from another class using extends keyword
// class Animal{
```

```

//      eat(){
//          console.log("eating")
//      }
// }
// class Dog extends Animal {
//      bark(){
//          console.log("barking")
//      }
// }
// const dog = new Dog();
// dog.eat()
// dog.bark()

//super keyword() => used to call the parent class constructor
// class User {
//      constructor(name){
//          this.name = name
//      }
// }
// class Admin extends User{
//      constructor (name, role){
//          super(name)
//          this.role = role
//      }
// }
// const admin = new Admin("Satyam","Admin")
// console.log(admin)

// method overriding or polymorphism
// class Animal{
//      sound(){
//          console.log("Animal sound")
//      }
// }
// class Dog extends Animal{
//      sound(){
//          console.log("Barking")
//      }
// }
// class Cat extends Animal{
//      sound(){
//          console.log("I'm cat")
//      }
// }

```

```

// const dog = new Dog()
// const cat = new Cat()
// dog.sound()
// cat.sound()

// Encapsulation => keeping data and related methods together
and controlling access to internal data
// js support private class using #
// class BankAccount{
//     #balance = 0
//     deposit(amount){
//         this.#balance = amount
//     }
//     getBalance(){
//         return this.#balance;
//     }
// }
// const sbi = new BankAccount()
// sbi.deposit(2000)
// console.log(sbi.getBalance())
// static method => Belongs to the class itself
// class MathHelper{
//     static add(a,b){
//         return a+b
//     }
// }
// console.log(MathHelper.add(10,20))

// abstraction => hide unnecessary implementation details and
expose only what needed
class User{
    #validateCredentials(password){
        return password === "Password123"
    }
    login(password){
        if(this.#validateCredentials(password)){
            console.log("Login ho gya")
        }
        else {
            console.log("Login nhi huaa")
        }
    }
}
const user1 = new User()

```

```
user1.login("Password123")
```